



Class and Object in JavaScript

1. What is a Class?

A **class** in JavaScript is a **blueprint** or **template** for creating objects.

It defines **properties** (data) and **methods** (functions) that the objects created from it will have.

Syntax:

```
class ClassName {  
    // properties  
    property1;  
    property2;  
  
    // method  
    methodName() {  
        // code  
    }  
}
```

2. What is an Object?

An **object** is an **instance** of a class.

You create an object using the **new** keyword.

Example:

```
class Car {  
    make;  
    model;  
  
    displayInfo() {
```

```
        console.log(`Car: ${this.make} ${this.model}`);
    }
}

// Create object
const myCar = new Car();
myCar.make = "Toyota";
myCar.model = "Corolla";
myCar.displayInfo(); // Output: Car: Toyota Corolla
```

Explanation:

- `Car` is the class (blueprint).
 - `myCar` is an object created from that blueprint.
 - Properties (`make`, `model`) are assigned after creating the object.
-

Constructor in JavaScript

3. What is a Constructor?

A **constructor** is a special method inside a class:

- It **automatically runs** when an object is created.
- Used to **initialize properties** of the object.

Syntax:

```
class ClassName {
    constructor(parameter1, parameter2) {
        // initialize properties
        this.property1 = parameter1;
        this.property2 = parameter2;
    }
}
```

Example (With Constructor):

javascript

CopyEdit

```
class Car {
    constructor(make, model) {
        this.make = make;
        this.model = model;
    }

    displayInfo() {
        console.log(`Car: ${this.make} ${this.model}`);
    }
}

// Create object with values
const myCar = new Car("Toyota", "Corolla");
myCar.displayInfo(); // Output: Car: Toyota Corolla
```

Explanation:

- `constructor(make, model)` takes values when the object is created.
- `this.make` and `this.model` store the values in the object.
- No need to assign properties separately after creating the object.

4. Key Differences Without vs With Constructor

Feature	Without Constructor	With Constructor
Property Assignment	After creating object	At the time of creation
Code Length	Slightly longer	Shorter & cleaner

Initialization

Manual

Automatic

✓ Summary:

- **Class** = Blueprint.
 - **Object** = Instance of the class.
 - **Constructor** = Special method to set values automatically when creating an object.
-

4 Pillars of Object-Oriented Programming (OOP) in JavaScript

The 4 Pillars are:

1. **Encapsulation** – Wrapping data & methods together and controlling access.
2. **Abstraction** – Hiding unnecessary details from the user.
3. **Inheritance** – Reusing properties & methods from another class.
4. **Polymorphism** – Same method name, different behaviors.

Encapsulation

Definition:

Encapsulation means **bundling data (properties)** and **methods (functions)** together inside a class, while restricting direct access to some data for safety and control.

Why #name?

In JavaScript, putting a # before a property makes it **private** — meaning it can **only** be

accessed inside the class.

This prevents outside code from changing it directly and keeps the data **safe**.

Example:

```
class Person {
  #name; // private property

  setName(name) {
    this.#name = name; // data is changed safely
  }

  getName() {
    return this.#name; // data is returned safely
  }
}

const p = new Person();
p.setName("John");
console.log(p.getName()); // John
// console.log(p.#name); ❌ Error - Direct access not allowed
```

✅ Key Point:

`#name` ensures no one can directly modify the property from outside the class.

All changes must go through the provided methods (`setName`, `getName`) — this is **encapsulation**.

Abstraction

Definition:

Abstraction means **hiding the complex internal logic** and showing only what is necessary to the user.

The user doesn't need to know *how* something works, just *how to use it*.

Why do it?

It makes code **simpler** for the user and prevents unnecessary access to complicated logic, reducing mistakes.

Example:

```
class CoffeeMachine {
  startMachine() {
    console.log("Starting the machine...");
  }

  brewCoffee() {
    console.log("Brewing coffee...");
  }

  makeCoffee() { // abstraction method
    this.startMachine();
    this.brewCoffee();
    console.log("Coffee is ready!");
  }
}

const myMachine = new CoffeeMachine();
myMachine.makeCoffee(); //  Easy for user
// Internals (startMachine & brewCoffee) are hidden
```

✓ Key Point:

The user just calls `makeCoffee()` — they don't care about the *steps inside* like starting the machine or brewing coffee.

Inheritance

📖 Definition:

Inheritance allows a class (**child**) to **reuse** properties and methods from another class (**parent**) so we don't have to write the same code again.

Why do it?

It makes code **reusable**, **organized**, and avoids duplication.

Example:

```
class Vehicle { // parent class
  move() {
    console.log("The vehicle is moving...");
  }
}

class Car extends Vehicle { // child class
  honk() {
    console.log("Beep beep!");
  }
}

const myCar = new Car();
myCar.move(); // ✓ Reused from Vehicle
myCar.honk(); // ✓ Specific to Car
```

✓ Key Point:

`Car` gets its own methods (`honk`) **and** inherits `move()` from `Vehicle` without rewriting it.

Polymorphism

Definition:

Polymorphism means **same method name, but different implementations** depending on the object that is using it.

Why do it?

It makes code **flexible** — you can use the same method name for different behaviors without changing the calling code.

Example:

```
class Bank {
    interestRate() {
        console.log("Generic bank interest rate");
    }
}

class SBI extends Bank {
    interestRate() {
        console.log("SBI Interest Rate: 6%");
    }
}

class HDFC extends Bank {
    interestRate() {
        console.log("HDFC Interest Rate: 7%");
    }
}
```



```
class ICICI extends Bank {
  interestRate() {
    console.log("ICICI Interest Rate: 6.5%");
  }
}

// Creating objects one by one
const sbiBank = new SBI();
sbiBank.interestRate(); // SBI Interest Rate: 6%

const hdfcBank = new HDFC();
hdfcBank.interestRate(); // HDFC Interest Rate: 7%

const iciciBank = new ICICI();
iciciBank.interestRate(); // ICICI Interest Rate: 6.5%
```

✅ **Key Point:**

The method `interestRate()` is the same, but each bank **overrides** it with its own details. This makes the same method work differently depending on the object.